ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH — TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

TÀI LIỆU ĐẶC TẢ KỸ THUẬT DỰ ÁN (PROJECT SPECIFICATION)

Đề tài: OpenLab-Scale — Quan sát tài nguyên Container mức Hạt nhân bằng eBPF và Quản lý thu hồi an toàn có phê duyệt (Monitoring-First Proof of Concept)

Mã & Tên dự án	OpenLab-Scale (Monitoring-First MVP Specification)
Học phần	Điện toán đám mây (Cloud Computing) — Học kỳ Fall 2026
Giảng viên hướng dẫn	PGS. Thoại Nam
Nhóm thực hiện (VYT Group)	Nguyễn Phúc Vĩnh – 2414001 | Trần Văn Minh Triết – 2413603 | Trần Đình Ý – 2414100
Phiên bản & Thời điểm	v1.0 (Chốt phạm vi Monitoring-First) — Tháng 10 / 2026

1. Tổng quan bài toán & Mục tiêu nghiên cứu

Trong các hạ tầng điện toán đám mây và phòng thí nghiệm ảo hóa bằng container (Cloud Lab / Cloud-Native), nhiều phiên làm việc chia sẻ chung nguồn tài nguyên phần cứng giới hạn trên máy chủ. Khi người dùng bỏ treo phiên hoặc tiến trình kết thúc pha tính toán nhưng vẫn giữ container sống, tài nguyên CPU/RAM bị chiếm dụng lãng phí. Tuy nhiên, việc tự động thu hồi container dựa trên các cơ chế truyền thống gặp ba hạn chế kỹ thuật lớn:

- **Fixed-Timeout (Ngắt theo thời gian cố định):** Áp đặt ngưỡng cắt phiên cứng nhắc, dễ tắt nhầm các tác vụ tính toán nền hợp lệ chạy dài hạn (batch jobs, biên dịch, huấn luyện mô hình) khi không có tương tác trực tiếp.
- **Polling cgroup / /proc chu kỳ thưa (15–30s):** Bỏ lọt các xung hoạt động ngắn (bursty blindness) và dễ mắc lỗi ngộ nhận nhàn rỗi (False-Idle): khi tiến trình chờ I/O đĩa, chờ lock hoặc đợi dữ liệu qua socket mạng, mức CPU tiệm cận 0% dù ứng dụng vẫn đang hoạt động tích cực.
- **In-Container / Sidecar Agent:** Đòi hỏi cài đặt tác tử vào từng container, gây tốn bộ nhớ phụ trội (footprint), xâm lấn môi trường khách và phụ thuộc chặt vào cấu trúc container image.

1.1. Mục tiêu của dự án OpenLab-Scale

OpenLab-Scale xây dựng một nguyên mẫu kiểm chứng (Proof of Concept) theo định hướng Monitoring-First: kết hợp quan sát sự kiện mức hạt nhân Linux bằng eBPF (phi xâm lấn từ tầng Host) với số liệu định lượng từ cgroup v2 để phân loại chính xác trạng thái hoạt động của container, đồng thời cung cấp quy trình thu hồi tài nguyên an toàn có kiểm duyệt của quản trị viên (Human-in-the-Loop) và cơ chế Fail-Safe khi suy giảm tín hiệu.

1.2. Câu hỏi nghiên cứu cốt lõi (Research Questions)

- **RQ1 — Độ tin cậy Telemetry & Ánh xạ:** Agent eBPF ở tầng host có thể thu thập sự kiện kernel và ánh xạ chính xác PID/TGID + cgroup sang Docker Container ID (kể cả khi tái sử dụng PID hoặc container khởi động lại) với tỷ lệ mất sự kiện kiểm soát được hay không?
- **RQ2 — Độ chính xác Phân loại:** Bộ phân loại đa tín hiệu (eBPF + cgroup v2) cải thiện Precision, Recall, F1 và giảm tỷ lệ False-Idle như thế nào so với các baseline Fixed-Timeout và CPU-only trên các kịch bản tải bursty / I/O-heavy?
- **RQ3 — Chi phí vận hành (Overhead):** Mức tiêu thụ CPU, RAM phụ trội và độ ổn định của eBPF collector trên máy chủ khi bật giám sát (Monitor-ON vs. Monitor-OFF) là bao nhiêu?
- **RQ4 — Tính an toàn khi Thu hồi:** Quy trình đề xuất Candidate-Idle kết hợp trạng thái UNKNOWN, phê duyệt của Admin, kiểm tra lại định danh (fencing check) và xác minh qua docker inspect có ngăn chặn triệt để hành vi dừng nhầm container khi dữ liệu trễ/lỗi hay không?

2. Phạm vi dự án (Bounded Scope)

Phân nhóm phạm vi	Chi tiết hạng mục kỹ thuật & Ranh giới hệ thống
Trong phạm vi bắt buộc (In-Scope / P0 MVP)	• Triển khai trên 01 máy chủ / máy ảo Ubuntu Linux kích hoạt cgroup v2 và kernel hỗ trợ BTF / eBPF. • Quản lý vòng đời trên Docker Engine (khởi chạy tái lập qua Docker Compose), quy mô 2–10 container đồng nhất hạn mức. • Bộ sinh tải giả lập có seed cố định (7 kịch bản) kèm xuất nhãn Ground-Truth độc lập theo thời gian. • Thu thập song song eBPF tracepoints và cgroup v2 CPU/RAM, gom cụm cửa sổ 5 giây có gắn nhãn nguồn gốc (provenance). • Bộ phân loại theo luật (ACTIVE / IDLE / MIXED / UNKNOWN) và Dashboard + Controller thu hồi bắt buộc Admin phê duyệt.
Ngoài phạm vi / Đã hoãn (Out-of-Scope / Deferred)	• Bộ cấp phát 100-seat Floating License, hàng đợi giữ chỗ và cơ chế lease/heartbeat (chuyển thành bối cảnh lịch sử, không chặn MVP). • Điều phối đa máy chủ (Multi-node cluster) hoặc tích hợp Kubernetes operator. • Tự động tắt/xóa container không có phê duyệt của người quản trị (Unattended auto-termination). • Giám sát nội dung phiên làm việc thật của sinh viên hoặc can thiệp vào bên trong container image.
Mở rộng tùy chọn (Optional / Stretch)	• Browser Input-Presence Adapter: ghi nhận tín hiệu hiện diện bàn phím/chuột giả lập (ngưỡng thử nghiệm 1/5/10/15 phút), trả về INTERACTION_UNKNOWN khi mất kết nối, tách biệt hoàn toàn với bộ đếm tài nguyên eBPF/cgroup.

3. Kiến trúc hệ thống & Đặc tả các thành phần chức năng

Hệ thống được thiết kế theo kiến trúc đường ống tách lớp trên một Ubuntu Docker Host, phân tách rõ luồng thu thập quan sát (Observability Plane), luồng ra quyết định & điều khiển có phê duyệt (Control Plane), và luồng đối chiếu thực nghiệm (Evaluation Plane):

Luồng kiến trúc tổng quát: [Synthetic Containers] → (Kernel Tracepoints / cgroupfs) → [eBPF Collector + cgroup v2 Sampler] → [Identity & 5s Telemetry Processor] → [Rule-Based Activity Detector] → [Resource Controller ⇄ Admin Dashboard] → (Approved Stop + Inspect Verify) → [Docker Engine]

Mô-đun hệ thống	Trách nhiệm xử lý kỹ thuật	Đầu ra & Giao diện dữ liệu
M1. Synthetic Workload Generator	Sinh tải Docker có kiểm soát theo 7 pha/kịch bản (CPU, I/O, Network, Bursty, Low-use, Sleep/Idle, Mixed) với random seed cố định.	Container chạy tải thực tế + Tệp nhãn Ground-Truth độc lập kèm timestamp (chỉ phục vụ Evaluation).
M2. eBPF Kernel Collector	Kiểm tra BTF/quyền kernel khi khởi động; gắn probe vào tracepoints tiến trình/lập lịch, VFS I/O và socket ở tầng host; đọc sự kiện qua BPF Ring Buffer.	Chuỗi sự kiện (timestamp, pid/tgid, cgroup_id, event_type) + bộ đếm sự kiện bị rớt (event_loss).
M3. cgroup v2 Sampler	Đọc định kỳ 5 giây các chỉ số định lượng CPU usage/quota và Memory bytes trực tiếp từ cây thư mục cgroup v2 của host.	Bản ghi định lượng gắn cờ provenance: source='cgroup_v2' (tách bạch khỏi sự kiện eBPF).
M4. Identity & Telemetry Processor	Ánh xạ (PID/TGID, cgroup_id) sang Docker Container ID; phát hiện PID reuse và container restart để xóa cache cũ; gom cụm theo cửa sổ trượt 5 giây.	JSON Telemetry mỗi 5s/container gồm: counters, resource deltas, telemetry_age, missing_flags, drop_count.
M5. Rule-Based Activity Detector	Đánh giá cửa sổ 5s để gán nhãn trạng thái theo luật minh bạch. Chu kỳ xét đề xuất mặc định 30s: yêu cầu ≥ 30s không hoạt động liên tục + 10s hysteresis.	Trạng thái {ACTIVE, IDLE, MIXED, UNKNOWN} kèm bằng chứng tín hiệu và độ tươi dữ liệu.

Mô-đun hệ thống	Trách nhiệm xử lý kỹ thuật	Đầu ra & Giao diện dữ liệu
M6. Resource Controller	Nhận trạng thái từ Detector để tạo đề xuất Candidate-Idle; chờ Admin phê duyệt; kiểm tra lại Container ID/trạng thái (fencing) trước khi gọi docker stop có timeout & tính lũy đẳng.	Lệnh dừng container an toàn + Xác minh trạng thái thật qua docker inspect + Bản ghi Audit Log.
M7. Monitoring Dashboard	Giao diện web/API trực quan hóa CPU, RAM, tín hiệu eBPF, tuổi telemetry, trạng thái phân loại, cảnh báo lỗi/dữ liệu cũ và nút bấm phê duyệt thu hồi.	Bảng điều hành thời gian thực, danh sách đề xuất thu hồi và lịch sử hành động (Action Audit).
M8. Evaluation Pipeline	Chạy lặp lại cùng seed trên 4 chính sách (No-reclaim, Fixed-timeout, CPU-only, Multi-signal eBPF+cgroup) và đối chiếu với nhãn Ground-Truth.	Confusion Matrix, Precision/Recall/F1, False-Idle rate, Overhead CPU/RAM, Reclaim latency.

4. Đặc tả trạng thái, Bất biến thiết kế & Cơ chế Fail-Safe

4.1. Tập trạng thái phân loại & Quy tắc chuyển trạng thái

- **ACTIVE:** Container có hoạt động CPU vượt ngưỡng hoặc phát sinh sự kiện tiến trình, đọc/ghi I/O, giao tiếp socket hợp lệ trong cửa sổ quan sát.
- **IDLE (và Candidate-Idle):** Cả số liệu định lượng cgroup v2 lẫn sự kiện định tính eBPF đều dưới ngưỡng tĩnh lặng liên tục trong 30 giây cộng thêm 10 giây trễ chống dao động (hysteresis).
- **MIXED:** Tín hiệu chuyển giao pha hoặc có hoạt động ngắt quãng/xung ngắn trong cửa sổ tổng hợp — hệ thống giữ nguyên container, không đề xuất thu hồi.
- **UNKNOWN (Trạng thái an toàn lỗi):** Kích hoạt ngay khi thiếu dữ liệu, telemetry quá hạn (stale), rớt sự kiện vượt ngưỡng, chưa ánh xạ được Container ID hoặc hai nguồn tín hiệu mâu thuẫn. Trạng thái UNKNOWN khóa cứng mọi đề xuất thu hồi.

4.2. Bốn bất biến an toàn của hệ thống (System Invariants)

- **Bất biến 1 — Tách biệt nguồn gốc (Provenance Separation):** Sự kiện eBPF và số liệu cgroup v2 luôn giữ nhãn nguồn gốc riêng trong schema JSON; phản hồi từ Docker Engine không bao giờ được dùng làm đặc trưng phân loại.
- **Bất biến 2 — Cách ly nhãn chuẩn (Ground-Truth Isolation):** Nhãn pha hoạt động từ bộ sinh tải chỉ được đọc ở bước chấm điểm sau thực nghiệm, tuyệt đối không đi vào Detector hay Controller.
- **Bất biến 3 — Chống định danh cũ (Anti-Stale Attribution):** Ánh xạ PID/cgroup → Container ID phải được đối chiếu với metadata vòng đời container; mọi sự kiện restart hoặc PID reuse phải làm mất hiệu lực ánh xạ cũ.
- **Bất biến 4 — Không thu hồi mù / tự động (No Unapproved or Degraded Reclaim):** Candidate-Idle không phải lệnh hủy tự động. Mọi thao tác dừng container bắt buộc thỏa đồng thời: (a) Telemetry tươi & hợp lệ, (b) Admin phê duyệt tường minh, (c) Khớp lại Container ID ngay trước khi thực thi.

4.3. Quy trình thu hồi tài nguyên chuẩn (6 bước)

(1) Thu thập sự kiện eBPF & mẫu cgroup v2 vào cửa sổ 5s → (2) Detector xuất trạng thái kèm bằng chứng và độ tươi dữ liệu → (3) Controller hiển thị đề xuất Candidate-Idle lên Dashboard → (4) Quản trị viên xem xét bằng chứng và bấm Phê duyệt (Approve) → (5) Controller kiểm tra lại (fencing recheck) Container ID, trạng thái sống và độ mới telemetry, sau đó gọi docker stop có timeout & khóa chống gọi trùng (idempotent) → (6) Gọi docker inspect xác nhận container đã dừng thực tế và ghi kết quả vào Audit Log.

4.4. Ma trận xử lý lỗi & Phản ứng Fail-Safe

Tình huống lỗi / Suy giảm (Failure Mode)	Phản ứng thiết kế bắt buộc (Expected Fail-Safe Response)

Tình huống lỗi / Suy giảm (Failure Mode)	Phản ứng thiết kế bắt buộc (Expected Fail-Safe Response)
Từ chối attach eBPF / Thiếu kernel BTF	Ghi log lỗi khởi tạo rõ ràng; kích hoạt chế độ báo cáo không hỗ trợ hoặc fallback cgroup-only có gắn cờ cảnh báo; tuyệt đối không giả lập thành công hay cho phép dừng container không an toàn.
Tràn Ring Buffer / Mất sự kiện (Event Loss)	Tăng bộ đếm event_loss và gắn cờ suy giảm chất lượng trên cửa sổ 5s; chuyển trạng thái sang UNKNOWN khi bằng chứng không đủ tin cậy và chặn đề xuất thu hồi.
Telemetry trễ / Mất kết nối Agent	Kiểm tra telemetry_age vượt ngưỡng tươi → gán lập tức trạng thái UNKNOWN trên Dashboard & Controller; vô hiệu hóa nút thu hồi cho đến khi nhận lại chuỗi cửa sổ khỏe mạnh.
Tái sử dụng PID / Container Restart	Phát hiện thay đổi cgroup/start-time của container; hủy cache ánh xạ cũ, phân giải lại Container ID mới và hủy mọi lệnh thu hồi đang chờ duyệt của phiên cũ (stale command rejection).
Lỗi phân quyền hoặc Timeout từ Docker API	Ghi nhận trạng thái thực thi thất bại vào Audit Log; gọi docker inspect để đồng bộ trạng thái thật của container; chỉ cho phép thử lại sau khi xác thực lại toàn bộ điều kiện.
Lệch đồng hồ / Nhập nhằng tại biên chuyển pha	Chuẩn hóa toàn bộ phép tính khoảng thời gian theo đồng hồ đơn điệu (monotonic clock); ghi nhận độ bất định tại vùng biên pha khi chấm điểm thực nghiệm.

5. Yêu cầu phi chức năng & Môi trường triển khai

- **Môi trường mục tiêu:** 01 máy chủ hoặc máy ảo Ubuntu Linux (LTS) bật cgroup v2 thống nhất, tracefs sẵn sàng, kernel hỗ trợ BTF, cài đặt Docker Engine và Docker Compose.
- **Đặc quyền tối thiểu (Least Privilege):** Tài liệu hóa tường minh các Linux capabilities tối thiểu cấp cho eBPF Agent ở tầng host (như CAP_BPF, CAP_PERFMON / quyền đọc tracefs và cgroupfs); không chỉnh sửa hay nâng quyền các container chạy tải.
- **Hiệu năng & Chi phí giám sát:** Xử lý lọc sự kiện ngay trong kernel và truyền qua Ring Buffer để giữ mức tiêu thụ phụ trội mục tiêu < 1.5% CPU host (được kiểm chứng thực nghiệm bằng phép đo đối chứng Monitor-ON và Monitor-OFF).
- **Tính tái lập (Reproducibility):** Toàn bộ hệ thống khởi động được từ một lệnh docker compose chuẩn hóa; mọi kịch bản tải đều dùng seed cố định; lưu trữ đầy đủ cấu hình phần cứng/kernel, dữ liệu thô và script vẽ biểu đồ.

6. Kế hoạch thực nghiệm, Chỉ số đo lường & Tiêu chí nghiệm thu

Hệ thống được đánh giá trên 7 kịch bản tải chuẩn hóa (CPU-steady, Disk I/O-heavy, Network-wait, Bursty short-pulse, Low-utilization active, Pure Sleep/Idle, và Mixed multi-phase) với cùng random seed chạy đối sánh ngang qua 4 cơ chế:

Cơ chế đối sánh	Nguyên tắc hoạt động	Mục đích kiểm chứng trong thực nghiệm
B0. No-Reclaim	Không thu hồi; giữ toàn bộ container chạy hết kịch bản.	Làm mốc tham chiếu mức tiêu thụ tài nguyên tối đa và kiểm chứng độ ổn định tải.
B1. Fixed-Timeout	Đánh dấu thu hồi sau một khoảng thời gian cố định không xét hành vi hạt nhân.	Đo tỷ lệ dừng nhầm các tác vụ nền chạy dài hạn.
B2. CPU-Only Polling	Chỉ dựa trên % CPU lấy mẫu định kỳ từ cgroup / docker stats.	Phơi bày lỗi False-Idle trên tải chờ I/O, chờ mạng và tải xung ngắn (bursty).
B3. OpenLab-Scale (Đề xuất)	Kết hợp đa tín hiệu eBPF Tracepoints + cgroup v2 (cửa sổ 5s, ngưỡng 30s + 10s hysteresis, Fail-Safe UNKNOWN).	Giảm thiểu False-Idle, bắt chính xác xung ngắn microsecond và đảm bảo thu hồi an toàn.

6.1. Bộ chỉ số đánh giá & Tiêu chí nghiệm thu MVP (Acceptance Criteria)

Hạng mục / RQ	Chỉ số đo lường định lượng (Metrics)	Tiêu chí nghiệm thu bắt buộc (Acceptance Criteria)
RQ1 — Telemetry & Attribution	Tỷ lệ ánh xạ đúng PID/cgroup → Container ID; số sự kiện rớt (event_loss); khả năng phát hiện PID reuse & restart.	Attach thành công ít nhất bộ tracepoint cốt lõi; xuất JSON 5s đúng schema; vượt qua bài test container restart & PID reuse.
RQ2 — Độ chính xác Phân loại	Confusion Matrix, Precision, Recall, F1-score, tỷ lệ False-Idle & False-Active, độ trễ phát hiện trạng thái.	Đối chiếu tự động với nhãn Ground-Truth độc lập trên đủ 7 kịch bản tải cùng seed; ghi nhận trung thực cả trường hợp sai số.
RQ3 — Overhead & Ổn định	Chênh lệch % CPU, dung lượng RAM trên host (Monitor-ON vs. OFF); tỷ lệ rớt sự kiện theo tải.	Có bộ script benchmark lặp lại được và log thô chứng minh agent chạy ổn định, không gây lỗi kernel hay treo Docker.
RQ4 — An toàn Thu hồi	Số lần dừng container trái phép / khi UNKNOWN; độ trễ thu hồi xác minh qua docker inspect; xử lý lệnh cũ (stale).	0 lần dừng container khi chưa có phê duyệt Admin hoặc khi telemetry thiếu/UNKNOWN; 100% lệnh dừng được verify bằng docker inspect.

6.2. Lộ trình triển khai theo 5 gói công việc (Epics)

Gói công việc	Tên Epic	Sản phẩm bàn giao chính (Key Deliverables)
Epic 1 (CC-9)	Research, Monitoring Scope & Architecture	Chốt đặc tả Monitoring-First MVP, kiến trúc luồng dữ liệu, ma trận Fail-Safe và môi trường Ubuntu/BTF.
Epic 2 (CC-10)	Docker Workload Simulation & Resource Control	Xây dựng bộ sinh 7 kịch bản tải có seed, xuất nhãn Ground-Truth độc lập và nguyên hàm điều khiển Docker an toàn.
Epic 3 (CC-11)	eBPF Container Observability	Hiện thực eBPF tracepoints, ánh xạ cgroup/PID → Container ID, lấy mẫu cgroup v2, gom cụm JSON 5s và đo overhead.
Epic 4 (CC-12)	Resource Analysis, Management & Dashboard	Hiện thực Detector theo luật, các baseline đối sánh, Controller có Admin phê duyệt + fencing check và Dashboard.
Epic 5 (CC-13)	Experiments, Seminar & Submission	Chạy thực nghiệm đối chứng, phân tích số liệu/biểu đồ, kiểm thử triển khai máy sạch (clean VM), hoàn thiện báo cáo & demo.